

Нейронные сети в машинном обучении

Генеративно-состязательные сети



Generative Models

The background of the slide features a series of overlapping, wavy, diagonal bands in two colors: a vibrant blue and a deep purple. These bands create a dynamic, geometric pattern that resembles a stylized, abstract landscape or a series of flowing ribbons. The text 'Generative Models' is positioned on the left side, with 'Generative' on the top line and 'Models' on the bottom line, both in a clean, white, sans-serif font.

Generative Models Hype



[Stable Diffusion - a Hugging Face Space](#)



[DALL·E mini by craiyon.com](#)

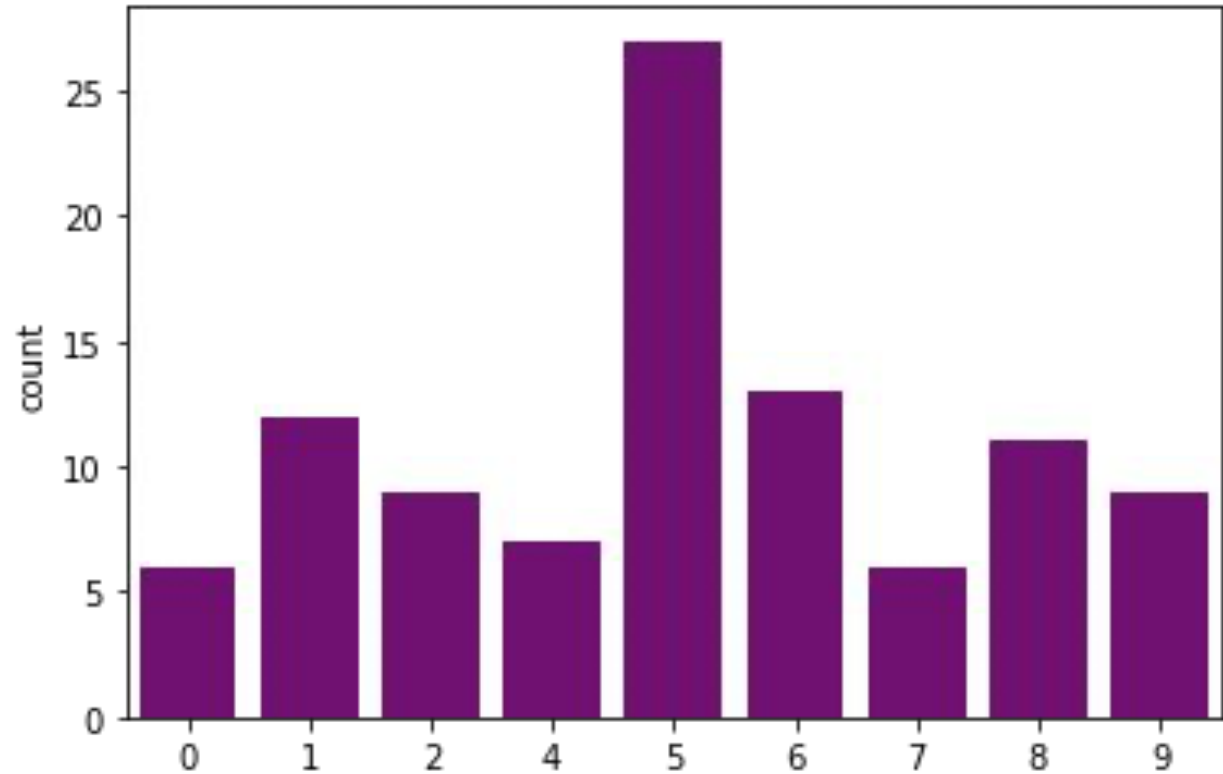


[This X Does Not Exist](#)



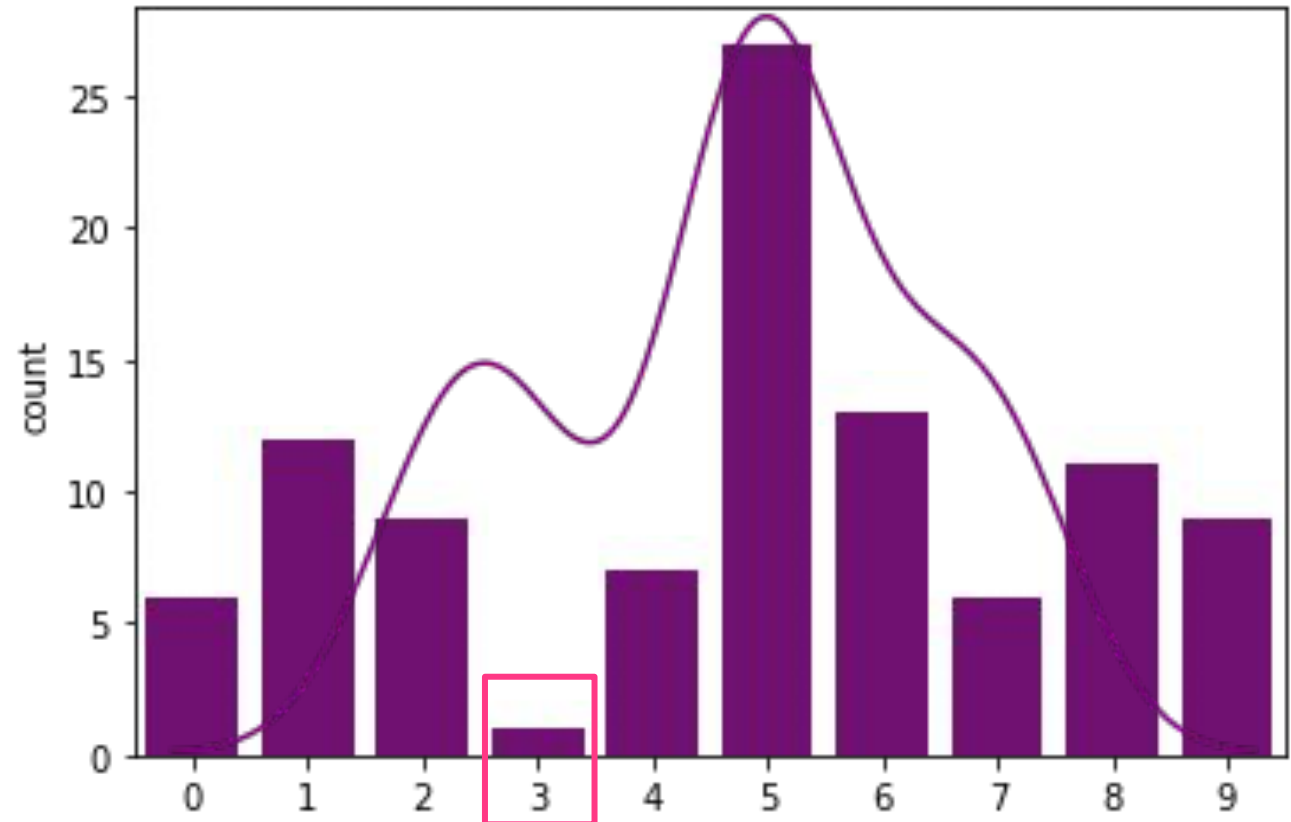
Toy Example

- We have an observed sample with numbers
- We want to generate a new object
- Let's assume there is a probability density $p(x)$
- We can try to estimate it using our sample and obtain $p_{\text{data}}(x)$
- Now we can sample from $p_{\text{data}}(x)$,
e.g. `np.random.choice(data)`
- **Do we face any problem here?**



Toy Example

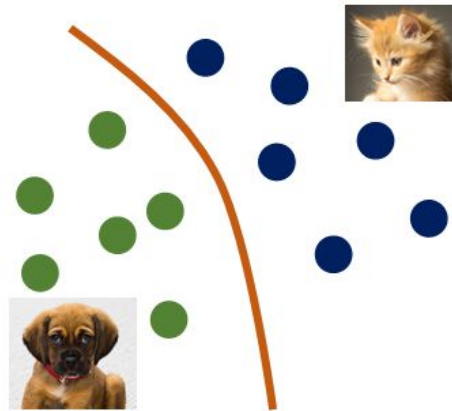
- We do not have “3” in our sample
- Common sense may help us
- Let’s pick up an **interpolation model**
- Now we can sample a new object that was not introduced into our data sample before



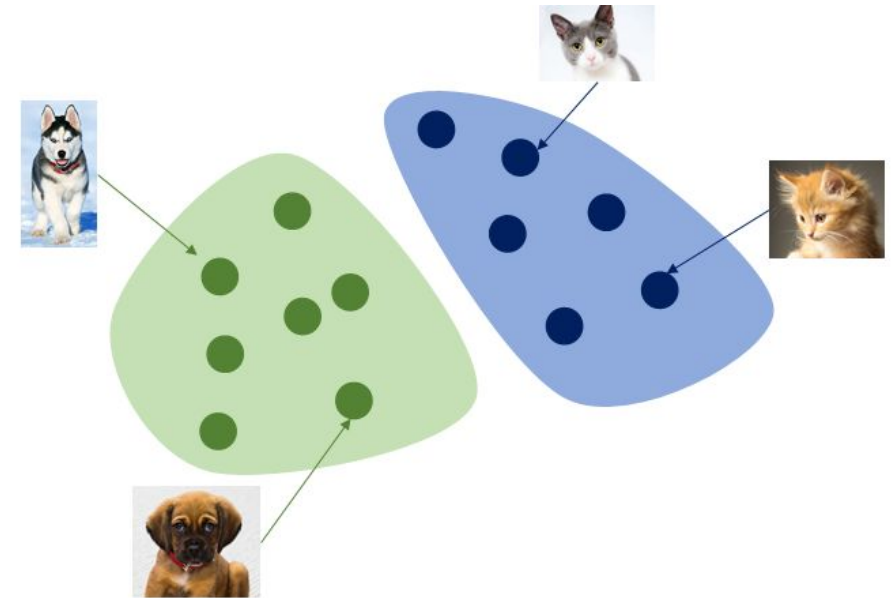
Discriminative VS Generative

For a given observations x and targets y :

- A discriminative model is a model of the conditional probability $\mathbf{P}(y|x)$
- A generative model is a model of the joint probability distribution $\mathbf{P}(x,y)$



Discriminative



Generative

Vanilla GAN

GAN Idea

- Let's assume that we have a **generator G** that samples from a random noise:
$$z \sim N(0;1)$$
$$x = G(z)$$
- G is an instance of NN
- Now we can sample generated objects and try to compare them to real ones
- You can check the batch of cats and give your predictions whether some particular image is real or not
- We can't use backprob in case of labels provided by ourselves, so let's build another network called **discriminator D** to distinguish between the real and generated samples.



Training Vanilla GAN. minimax game

Discriminator

- Classification setting
- Learns to map object into labels (real, fake)
- $D(x)$ represents the probability that x came from the data rather than it was generated
- D maximizes the probability of assigning the correct label to both training examples and samples from G
- $D(G(z)) = 0$

Generator

- Reversed bottleneck architecture
- $G(z)$ outputs objects we want to generate
- Tries to trick Discriminator D
- $D(G(z)) = 1$

D and G play the following two-player minimax game with value function $V(G, D)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Training Vanilla GAN. Algorithm

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log \left(1 - D(G(z^{(i)})) \right) \right].$$

end for

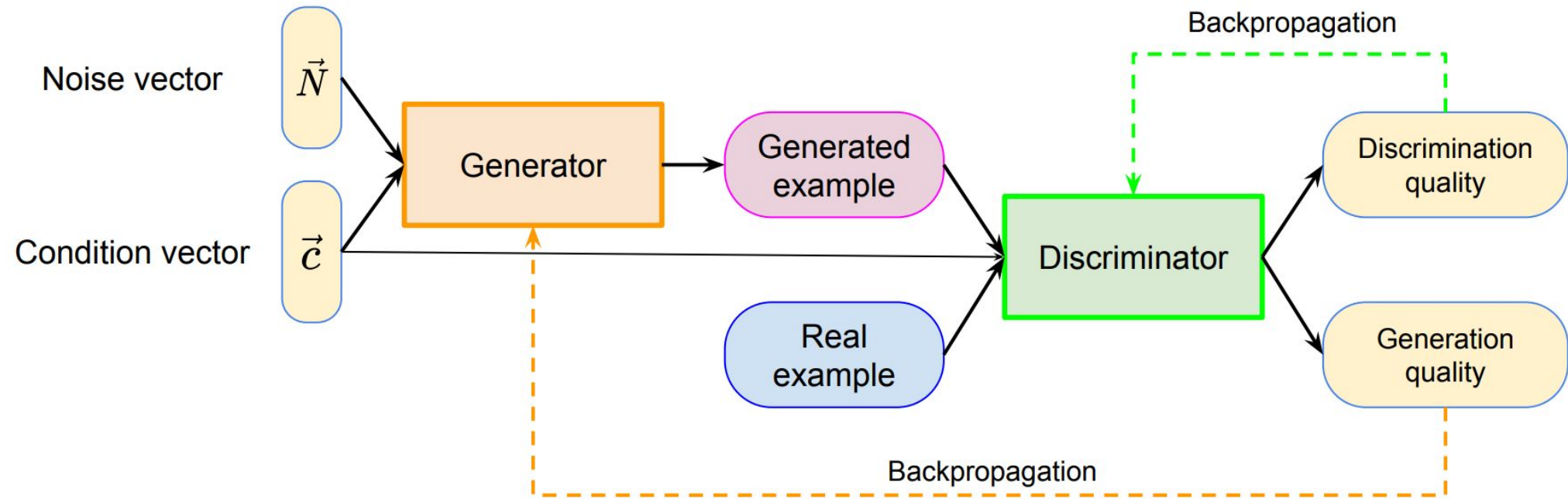
- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D(G(z^{(i)})) \right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

GAN conceptual example



Vanilla GAN Global Optimality

- Let's rewrite V using definition of expected value of continuous random variable
- Perform transition from z to x as $x = G(z)$
(*Law of the unconscious statistician*)
- Discriminator maximizes this expression, so let's find it's optimum
- Fix G
- For any $(a, b) \in \mathbb{R}^2 \setminus \{0, 0\}$, the function
 $y \rightarrow a \log(y) + b \log(1 - y)$ achieves its maximum in $[0, 1]$ at $\frac{a}{a+b}$
- Use this expression to get $D^*(x)$

$$\begin{aligned} V(G, D) &= \int_{\mathbf{x}} p_{\text{data}}(\mathbf{x}) \log(D(\mathbf{x})) d\mathbf{x} + \int_{\mathbf{z}} p_{\mathbf{z}}(\mathbf{z}) \log(1 - D(g(\mathbf{z}))) d\mathbf{z} \\ &= \int_{\mathbf{x}} p_{\text{data}}(\mathbf{x}) \log(D(\mathbf{x})) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x})) d\mathbf{x} \end{aligned}$$

$$D_G^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}$$

Vanilla GAN Global Optimality

- Recap KL and JSD
- Think of Jensen-Shannon divergence like symmetric KL divergence
- Let $D = D^*(x)$
- If $p_{data}(x) = p_g(x)$, $D^*(x) = 0.5$
- Thus $V(G, D^*) = C(G) = \log(0.5) + \log(0.5) = -\log 4$
- JSD between two distributions is always non-negative and zero only when they are equal,
- $C(G) = -\log(4)$ is the global minimum of and that the only solution
- GAN perfectly replicating the data generating process

$$KL(\pi || p) = \int \pi(\mathbf{x}) \log \frac{\pi(\mathbf{x})}{p(\mathbf{x}|\theta)} d\mathbf{x}$$

$$JSD(\pi(\mathbf{x}) || p(\mathbf{x}|\theta)) = \frac{1}{2} \left[KL \left(\pi(\mathbf{x}) || \frac{\pi(\mathbf{x}) + p(\mathbf{x}|\theta)}{2} \right) + KL \left(p(\mathbf{x}|\theta) || \frac{\pi(\mathbf{x}) + p(\mathbf{x}|\theta)}{2} \right) \right]$$

$$\begin{aligned} C(G) &= \max_D V(G, D) \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_G^*(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D_G^*(G(\mathbf{z})))] \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_G^*(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p_g} [\log(1 - D_G^*(\mathbf{x}))] \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} \left[\log \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_g(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right] \end{aligned}$$

$$C(G) = -\log(4) + KL \left(p_{data} \left\| \frac{p_{data} + p_g}{2} \right\| \right) + KL \left(p_g \left\| \frac{p_{data} + p_g}{2} \right\| \right)$$

$$C(G) = -\log(4) + 2 \cdot JSD(p_{data} || p_g)$$

Vanilla GAN

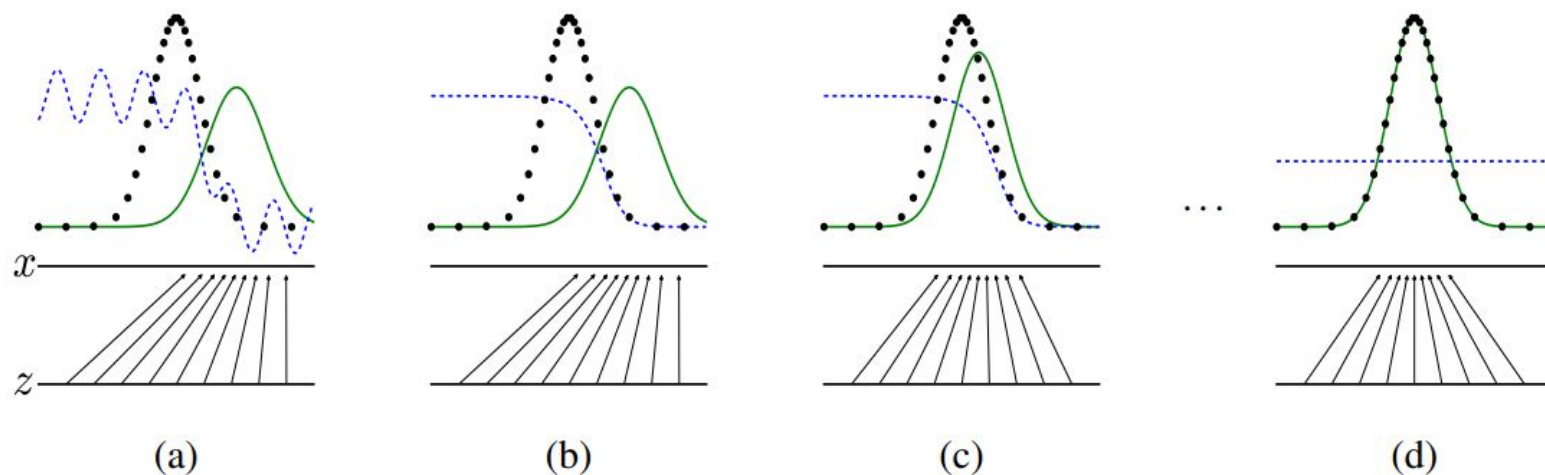


Figure 1: Generative adversarial nets are trained by simultaneously updating the discriminative distribution (D , blue, dashed line) so that it discriminates between samples from the data generating distribution (black, dotted line) p_x from those of the generative distribution p_g (G) (green, solid line). The lower horizontal line is the domain from which z is sampled, in this case uniformly. The horizontal line above is part of the domain of x . The upward arrows show how the mapping $x = G(z)$ imposes the non-uniform distribution p_g on transformed samples. G contracts in regions of high density and expands in regions of low density of p_g . (a) Consider an adversarial pair near convergence: p_g is similar to p_{data} and D is a partially accurate classifier. (b) In the inner loop of the algorithm D is trained to discriminate samples from data, converging to $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$. (c) After an update to G , gradient of D has guided $G(z)$ to flow to regions that are more likely to be classified as data. (d) After several steps of training, if G and D have enough capacity, they will reach a point at which both cannot improve because $p_g = p_{\text{data}}$. The discriminator is unable to differentiate between the two distributions, i.e. $D(x) = \frac{1}{2}$.

Vanilla GAN Results

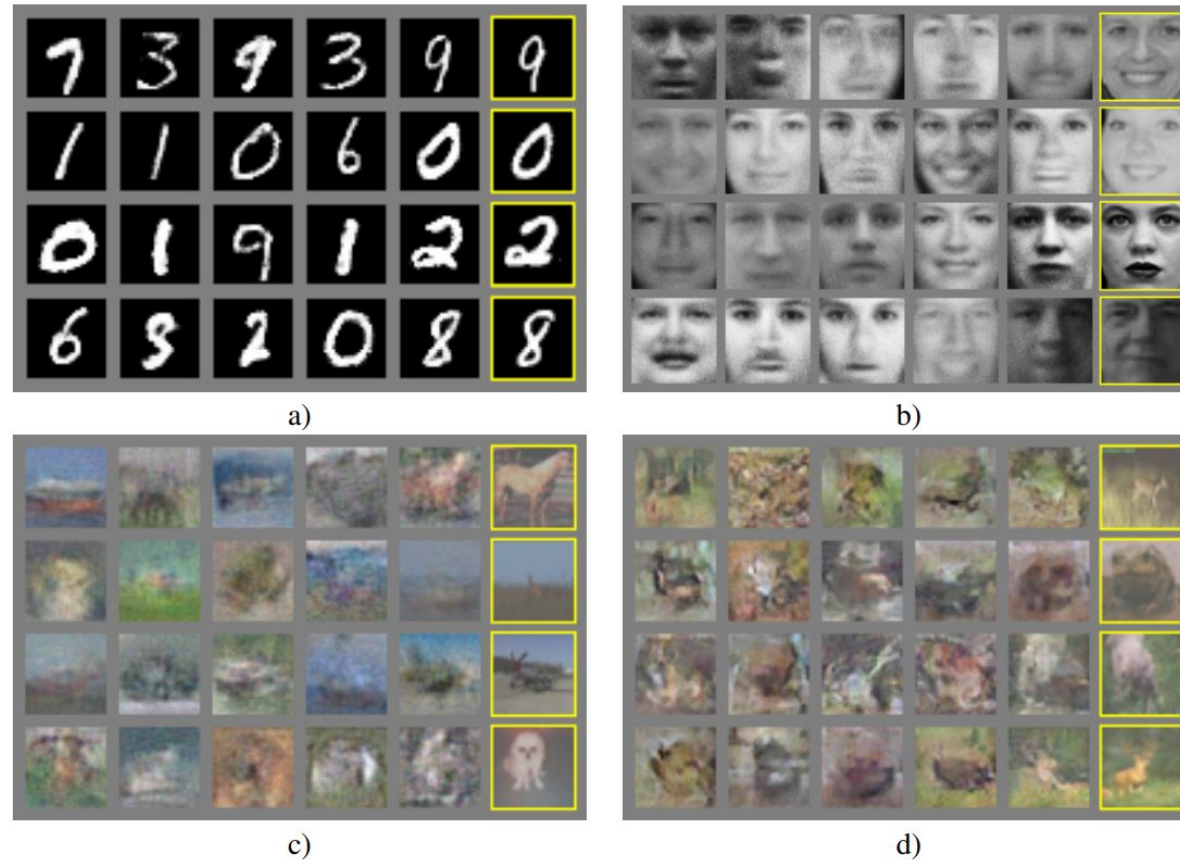


Figure 2: Visualization of samples from the model. Rightmost column shows the nearest training example of the neighboring sample, in order to demonstrate that the model has not memorized the training set. Samples are fair random draws, not cherry-picked. Unlike most other visualizations of deep generative models, these images show actual samples from the model distributions, not conditional means given samples of hidden units. Moreover, these samples are uncorrelated because the sampling process does not depend on Markov chain mixing. a) MNIST b) TFD c) CIFAR-10 (fully connected model) d) CIFAR-10 (convolutional discriminator and “deconvolutional” generator)

GAN Problems

The background of the slide features a series of thick, wavy, diagonal stripes in two colors: a vibrant pink and a deep purple. The stripes create a dynamic, rhythmic pattern that flows across the entire frame. The text 'GAN Problems' is centered in the upper half of the image, rendered in a clean, white, sans-serif font.

Vanishing gradients

- During early epochs G is weak
- D can reject samples with high confidence
- $\log(1 - D(G(z)))$ saturates

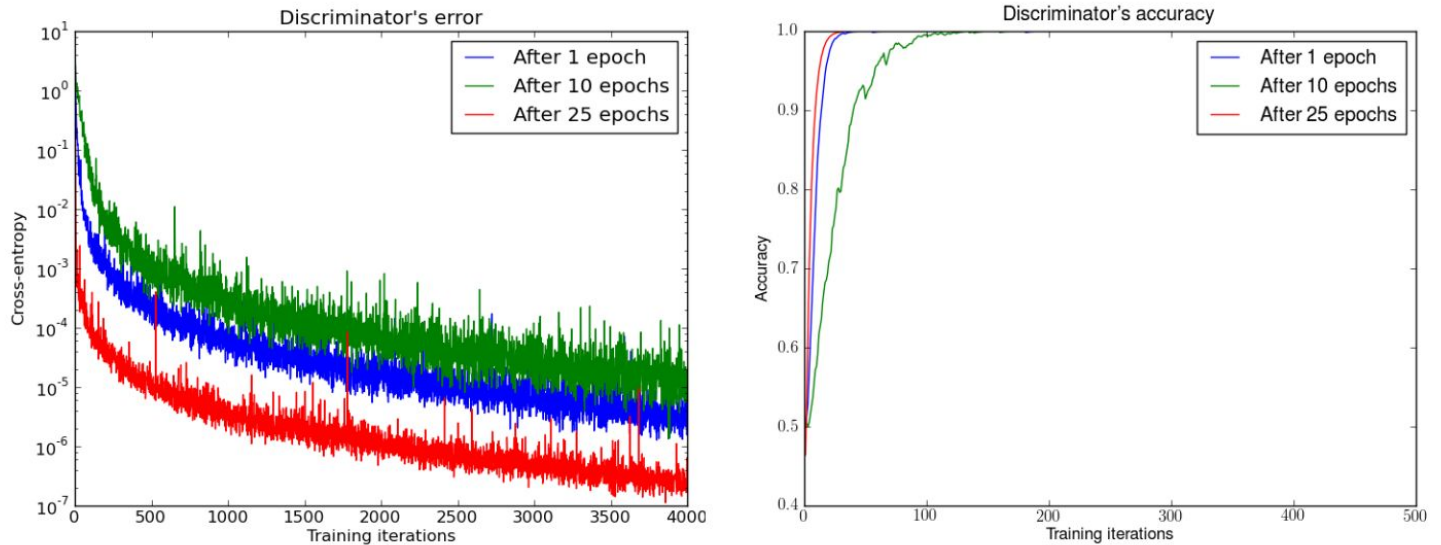
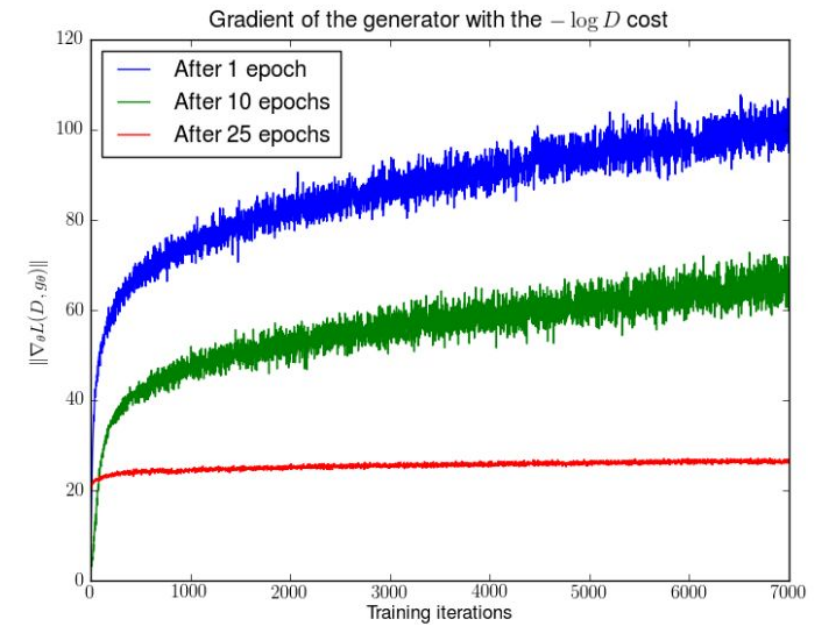
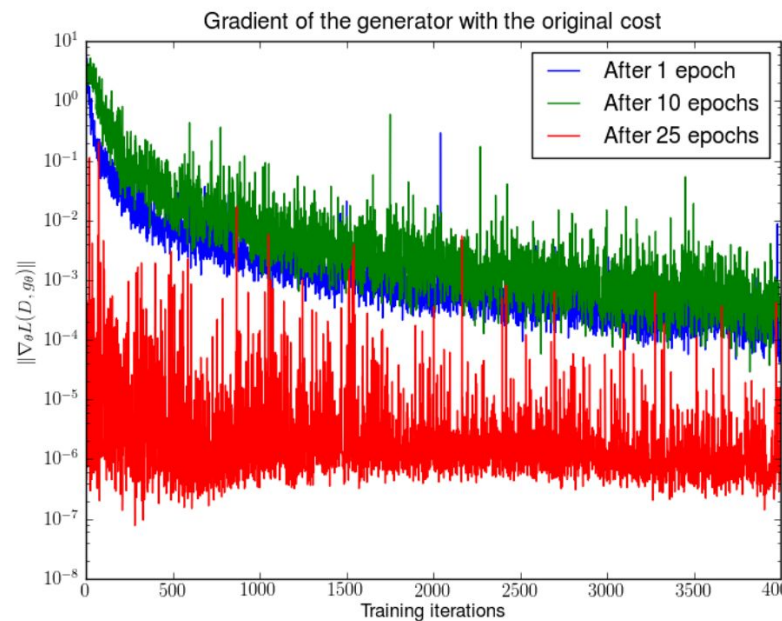


Figure 1: First, we trained a DCGAN for 1, 10 and 25 epochs. Then, with the generator fixed we train a discriminator from scratch. We see the error quickly going to 0, even with very few iterations on the discriminator. This even happens after 25 epochs of the DCGAN, when the samples are remarkably good and the supports are likely to intersect, pointing to the non-continuity of the distributions. Note the logarithmic scale. For illustration purposes we also show the accuracy of the discriminator, which goes to 1 in sometimes less than 50 iterations. This is 1 even for numerical precision, and the numbers are running averages, pointing towards even faster convergence.

Vanishing gradients

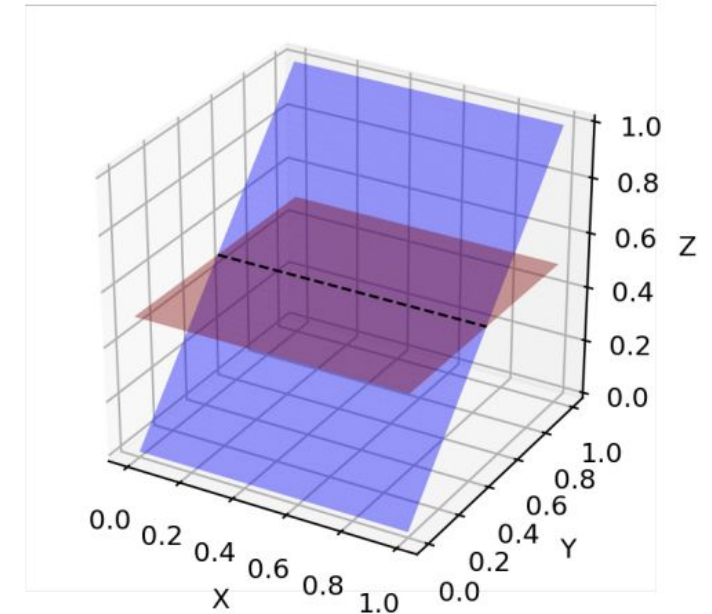
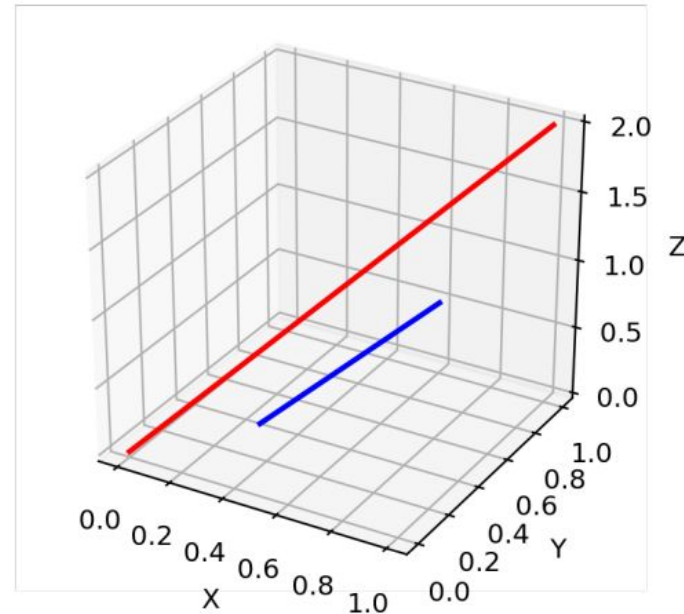
- Maximize $\log D(G(z))$ instead of minimizing $\log(1 - D(G(z)))$
- While new version of gradient doesn't necessarily suffer from vanishing gradients, it does cause massively unstable updates



[Arjovsky M., Bottou L. Towards principled methods for training generative adversarial networks //arXiv preprint arXiv:1701.04862. – 2017.](#)

Low Dimensional Supports

- Manifold - a topological space that locally resembles Euclidean space near each point
- Support - A real-valued function f is the subset of the domain containing those elements which are not mapped to zero.
- The dimensions of many real-world datasets only appear to be artificially high.
- Once the theme or the contained object is fixed, the images have a lot of restrictions to follow, i.e., a dog should have two ears and a tail, and a skyscraper should have a straight and tall body, etc.
- These restrictions keep images away from the possibility of having a high-dimensional free form.
- Whenever the generator is asked to a much larger image given a small dimension noise variable input, the distribution of colors over these pixels has been defined by the small random number vector and can hardly fill up the whole high dimensional space.
- Because both p_g and p_r rest in low dimensional manifolds, they are almost certainly disjoint.
- When they have disjoint supports, we are always capable of finding a perfect discriminator that separates real and fake samples correctly



Low dimensional manifolds in high dimension space can hardly have overlaps. (Left) Two lines in a three-dimension space. (Right) Two surfaces in a three-dimension space.

Mode collapse

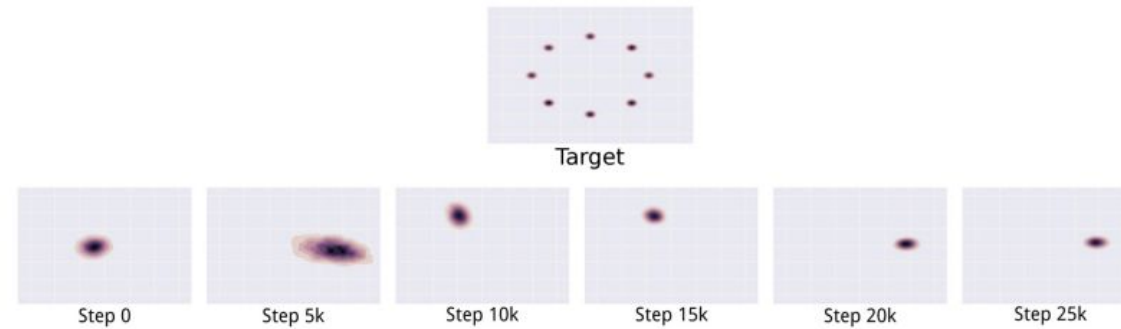


Figure 22: An illustration of the mode collapse problem on a two-dimensional toy dataset. In the top row, we see the target distribution p_{data} that the model should learn. It is a mixture of Gaussians in a two-dimensional space. In the lower row, we see a series of different distributions learned over time as the GAN is trained. Rather than converging to a distribution containing all of the modes in the training set, the generator only ever produces a single mode at a time, cycling between different modes as the discriminator learns to reject each one.

Mode collapse. DCGAN

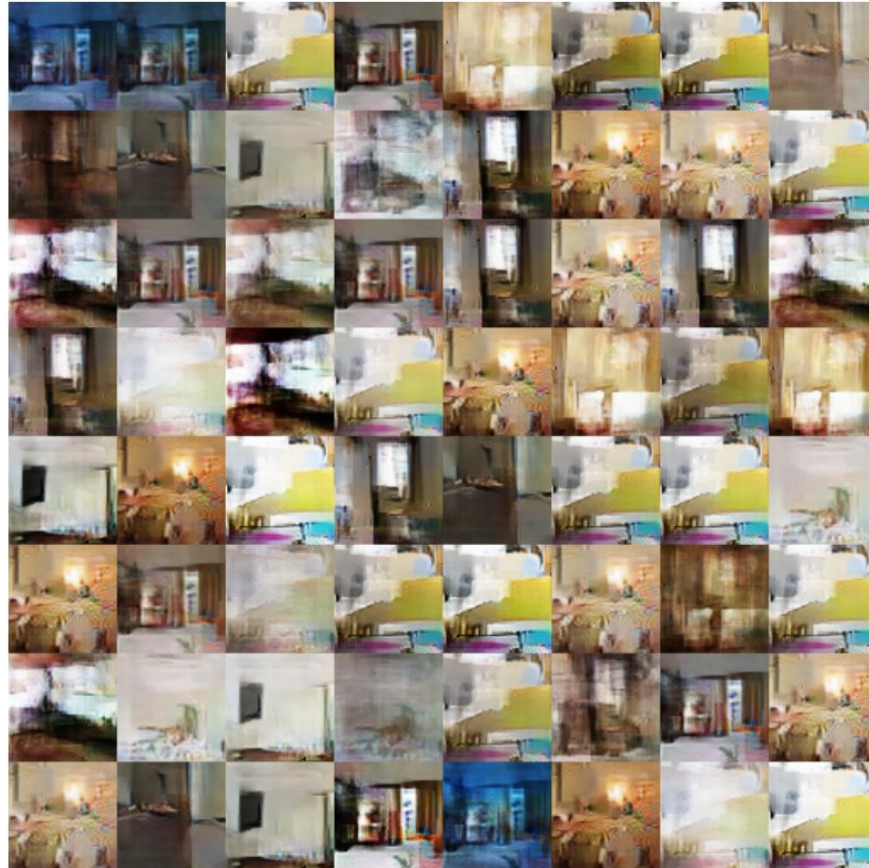
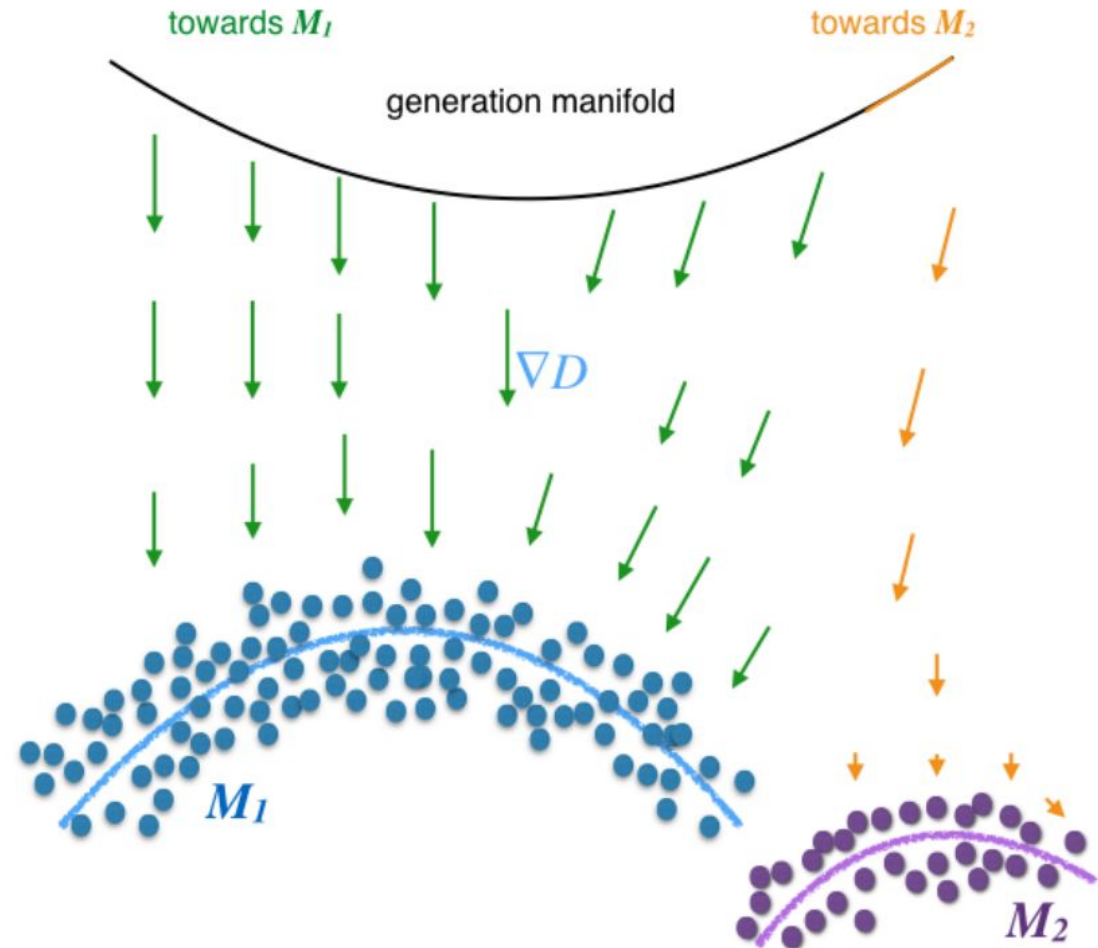


Figure 14: Standard GAN procedure: generator is an MLP with 4 hidden layers of 512 units, discriminator is a DCGAN.

[Wasserstein GAN](#)

Mode collapse

- For most z , the gradient of the generator pushes the generator towards the major mode M_1 to fool D the most
- G may even become independent on z
- Only when $G(z)$ is very close to the mode M_2 the generator can get gradients to push itself towards the minor mode M_2
- As D restarts, it easily finds objects, corresponding to M_1
- G finds new mode



GAN Improvements

LSGAN

- LSGANs adopts least squares loss function for the discriminator.
- Mao X. et al show that minimizing the objective function of LSGAN yields minimizing the Pearson χ^2 divergence.
- a-b coding scheme for the discriminator, where a and b are the labels for fake data and real data, respectively.
- c denotes the value that G wants D to believe for fake data

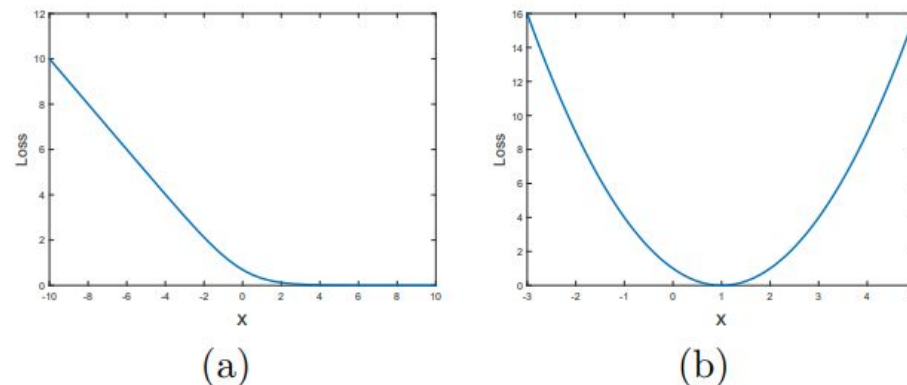


Figure 2: (a): The sigmoid cross entropy loss function. (b): The least squares loss function.

$$\min_D V_{\text{LSGAN}}(D) = \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [(D(\mathbf{x}) - b)^2] + \frac{1}{2} \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [(D(G(\mathbf{z})) - a)^2]$$
$$\min_G V_{\text{LSGAN}}(G) = \frac{1}{2} \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [(D(G(\mathbf{z})) - c)^2],$$

[Mao X. et al. Least squares generative adversarial networks //Proceedings of the IEEE international conference on computer vision. – 2017. – C. 2794-2802.](#)

LSGAN. Toy example

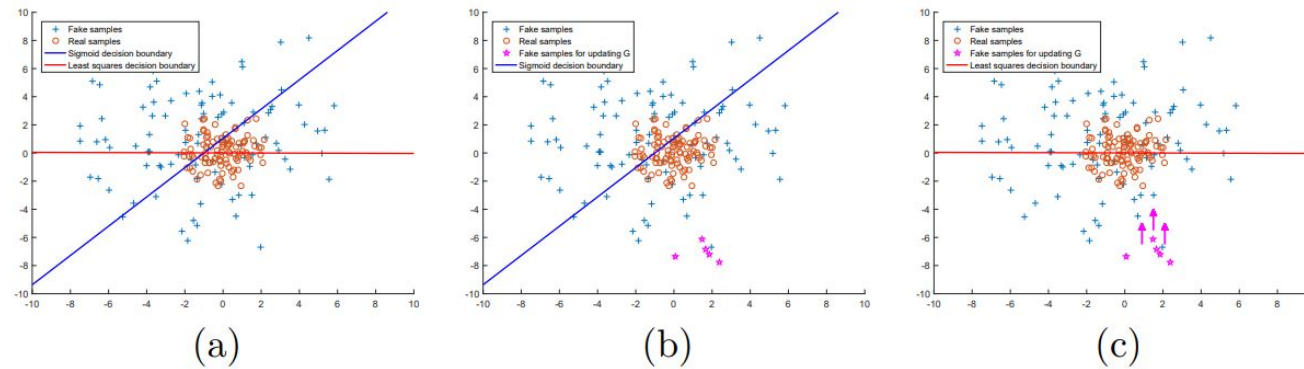
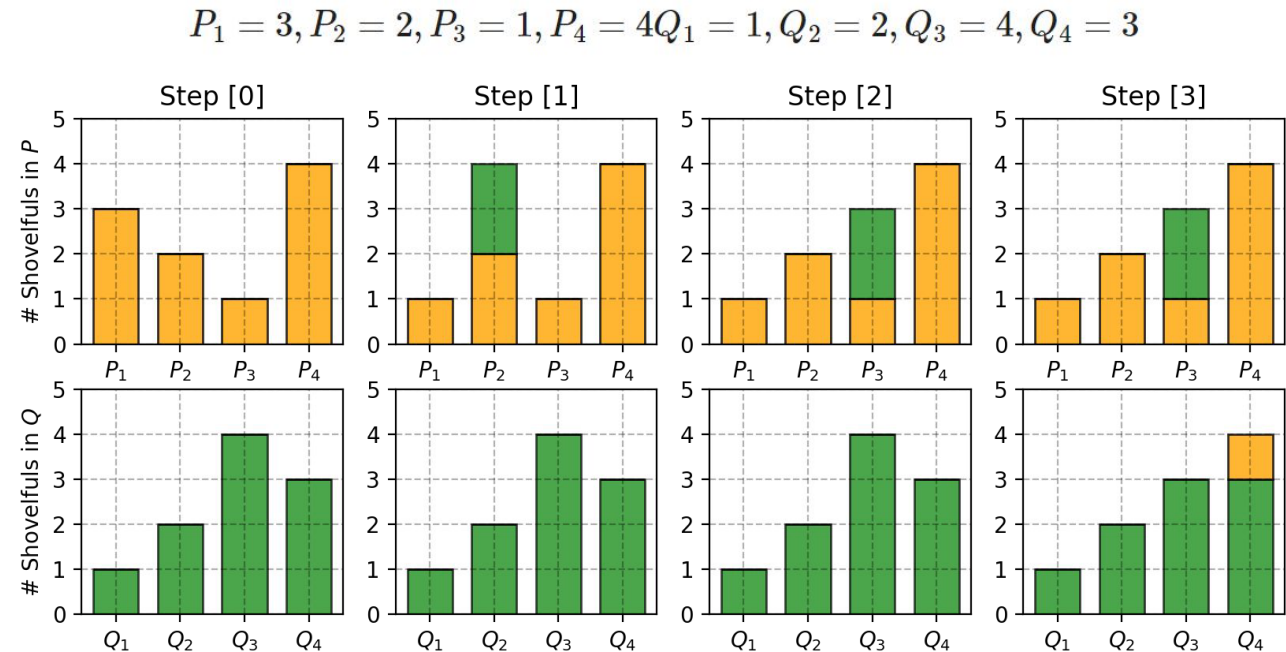


Figure 1: Illustration of different behaviors of two loss functions. (a): Decision boundaries of two loss functions. Note that the decision boundary should go across the real data distribution for a successful GANs learning. Otherwise, the learning process is saturated. (b): Decision boundary of the sigmoid cross entropy loss function. It gets very small errors for the fake samples (in magenta) for updating G as they are on the correct side of the decision boundary. (c): Decision boundary of the least squares loss function. It penalize the fake samples (in magenta), and as a result, it forces the generator to generate samples toward decision boundary.

[Mao X. et al. Least squares generative adversarial networks //Proceedings of the IEEE international conference on computer vision. – 2017. – C. 2794-2802.](#)

Wasserstein distance

- Wasserstein (Earth Mover's) Distance is a measure of the distance between two probability distributions
- Informally it can be interpreted as the minimum energy cost of moving and transforming a pile of dirt in the shape of one probability distribution to the shape of the other distribution
- The cost is quantified by: the amount of dirt moved multiplied by the moving distance



- First move 2 shovelfuls from P_1 to $P_2 \Rightarrow (P_1, Q_1)$ match up.
- Then move 2 shovelfuls from P_2 to $P_3 \Rightarrow (P_2, Q_2)$ match up.
- Finally move 1 shovelfuls from Q_3 to $Q_4 \Rightarrow (P_3, Q_3)$ and (P_4, Q_4) match up.

$$WD(P, Q) = 2 \times \text{step}(1,2) + 2 \times \text{step}(2,3) + 1 \times \text{step}(3,4) = 5$$

WD Continuous

When dealing with the continuous probability domain, the distance formula becomes:

$$W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$$

In the formula above, $\Pi(p_r, p_g)$ is the set of all possible joint probability distributions between p_r and p_g . One joint distribution $\gamma \in \Pi(p_r, p_g)$ describes one dirt transport plan, same as the discrete example above, but in the continuous probability space. Precisely $\gamma(x, y)$ states the percentage of dirt should be transported from point x to y so as to make x follows the same probability distribution of y . That's why the marginal distribution over x adds up to p_g , $\sum_x \gamma(x, y) = p_g(y)$ (Once we finish moving the planned amount of dirt from every possible x to the target y , we end up with exactly what y has according to p_g .) and vice versa $\sum_y \gamma(x, y) = p_r(x)$.

When treating x as the starting point and y as the destination, the total amount of dirt moved is $\gamma(x, y)$ and the travelling distance is $|x - y|$ and thus the cost is $\gamma(x, y) \cdot |x - y|$. The expected cost averaged across all the (x, y) pairs can be easily computed as:

$$\sum_{x,y} \gamma(x, y) \|x - y\| = \mathbb{E}_{x,y \sim \gamma} \|x - y\|$$

Finally, we take the minimum one among the costs of all dirt moving solutions as the EM distance.

WD. Toy example

$\forall (x, y) \in P, x = 0 \text{ and } y \sim U(0, 1) \quad \forall (x, y) \in Q, x = \theta, 0 \leq \theta \leq 1 \text{ and } y \sim U(0, 1)$

When $\theta \neq 0$:

$$D_{KL}(P\|Q) = \sum_{x=0, y \sim U(0,1)} 1 \cdot \log \frac{1}{0} = +\infty$$

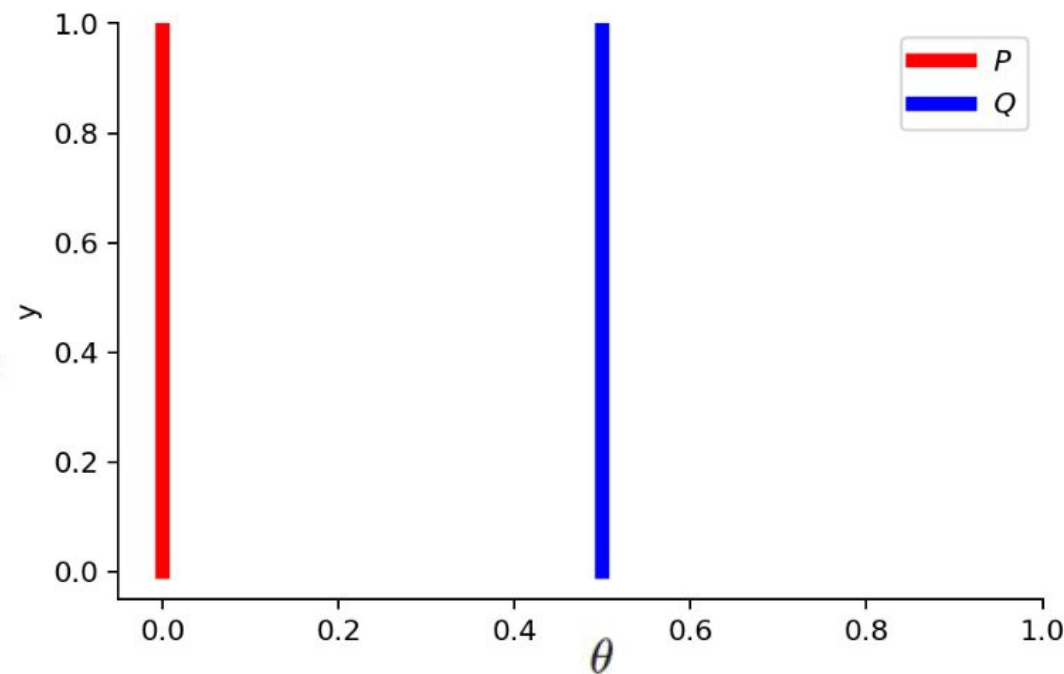
$$D_{KL}(Q\|P) = \sum_{x=\theta, y \sim U(0,1)} 1 \cdot \log \frac{1}{0} = +\infty$$

$$D_{JS}(P, Q) = \frac{1}{2} \left(\sum_{x=0, y \sim U(0,1)} 1 \cdot \log \frac{1}{1/2} + \sum_{x=\theta, y \sim U(0,1)} 1 \cdot \log \frac{1}{1/2} \right) = \log 2$$

$$W(P, Q) = |\theta|$$

But when $\theta = 0$, two distributions are fully overlapped:

$$D_{KL}(P\|Q) = D_{KL}(Q\|P) = D_{JS}(P, Q) = 0$$
$$W(P, Q) = 0 = |\theta|$$



WGAN

- It is intractable to exhaust all the possible joint distributions to compute infimum (greatest lower bound)
- Arjovsky et al. proposed a transformation of the formula based on the Kantorovich-Rubinstein duality
- Now we want to measure the least upper bound
- In the modified Wasserstein-GAN, the “discriminator” is trained to learn a function to help compute Wasserstein distance.
- As the loss function decreases in the training, the Wasserstein distance gets smaller and the generator model’s output grows closer to the real data distribution.
- Our discriminator (critic) should be K-Lipschitz continuous.
- If you are interested in how to transfer Wasserstein metric into its dual form according to the Kantorovich-Rubinstein Duality, read this [post](#).

[From GAN to WGAN](#)

A real-valued function $f : \mathbb{R} \rightarrow \mathbb{R}$ is called K -Lipschitz continuous if there exists a real constant $K \geq 0$ such that, for all $x_1, x_2 \in \mathbb{R}$,

$$|f(x_1) - f(x_2)| \leq K|x_1 - x_2|$$

Transformed WD:

$$W(p_r, p_g) = \frac{1}{K} \sup_{\|f\|_L \leq K} \mathbb{E}_{x \sim p_r}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)]$$

WGAN Objective:

$$L(p_r, p_g) = W(p_r, p_g) = \max_{w \in W} \mathbb{E}_{x \sim p_r}[f_w(x)] - \mathbb{E}_{z \sim p_r(z)}[f_w(g_\theta(z))]$$

WGAN. Algorithm

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α , the learning rate. c , the clipping parameter. m , the batch size. n_{critic} , the number of iterations of the critic per generator iteration.

Require: : w_0 , initial critic parameters. θ_0 , initial generator's parameters.

```
1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w \left[ \frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)})) \right]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while
```

WGAN VS Vanilla-GAN

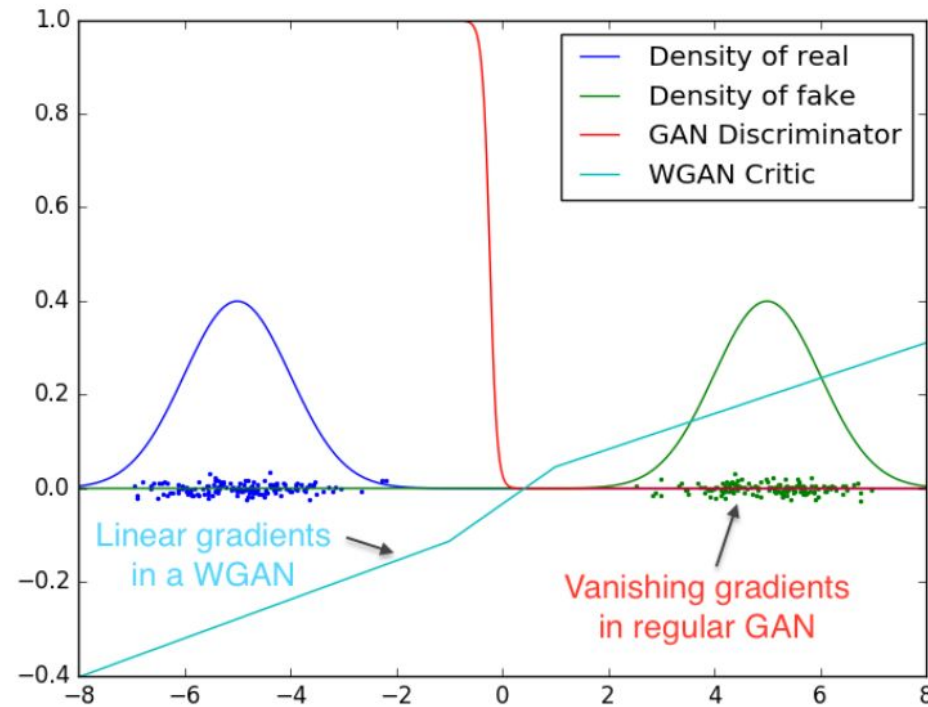
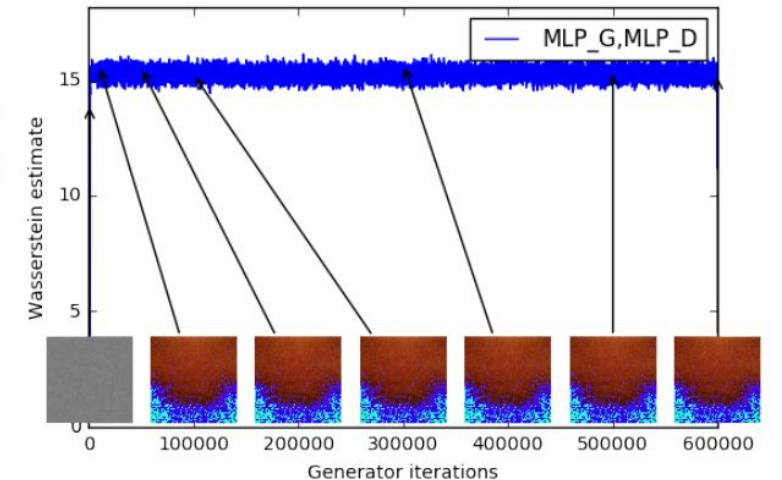
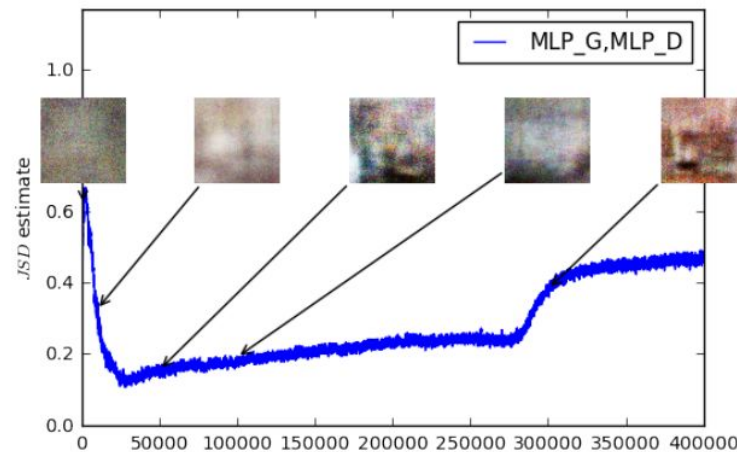
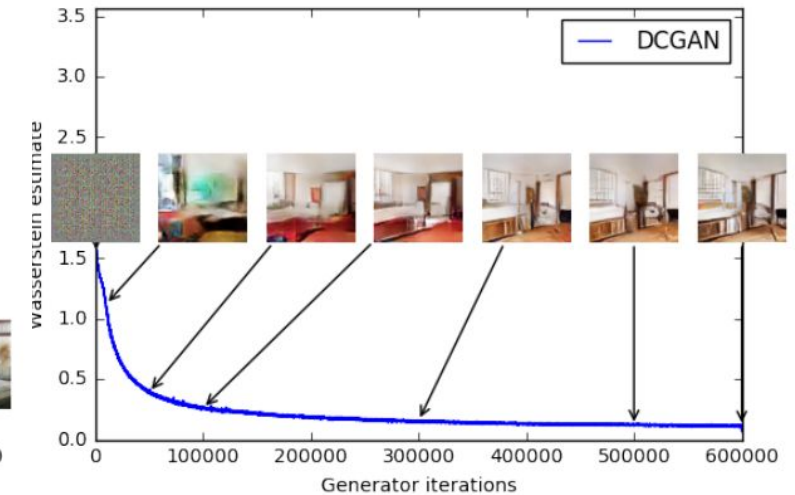
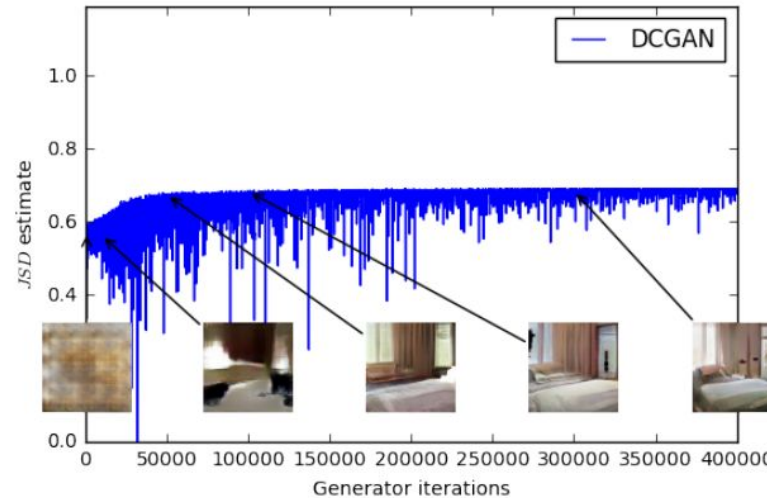


Figure 2: Optimal discriminator and critic when learning to differentiate two Gaussians. As we can see, the discriminator of a minimax GAN saturates and results in vanishing gradients. Our WGAN critic provides very clean gradients on all parts of the space.

WD-loss vs JS-loss

- JS:
 - Samples get better for the DCGAN but the JS estimate increases or stays constant, pointing towards no significant correlation between sample quality and loss
 - MLP curve goes up and down regardless of sample quality
- WD:
 - Correlation between lower error and better sample quality in case of WGAN
 - Loss is constant and samples are constant as well in case of WGAN-MLP



Lipschitz-1 Condition

- Original paper used Weight Clipping:

$$w \leftarrow \text{clip}(w, -c, c)$$

- Weight clipping makes the critic less expressive and the training harder to converge

- A gradient penalty term can be added into the loss with the weight λ :

$$\mathbb{E}_{\hat{\mathbf{x}} \sim \mathbb{P}_{\hat{\mathbf{x}}}} [(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2]$$

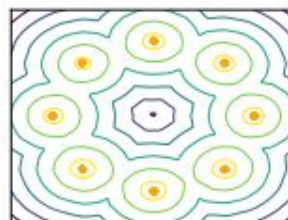
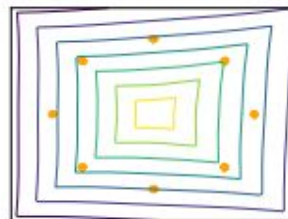
- We can normalize weights using spectral normalization:

$$\bar{W}_{\text{SN}}(W) := W / \sigma(W)$$

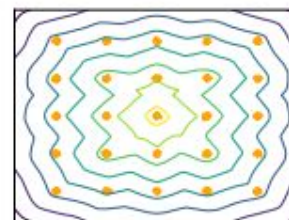
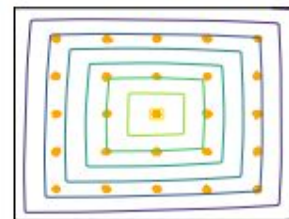
$$\sigma(A) := \max_{\mathbf{h}: \mathbf{h} \neq \mathbf{0}} \frac{\|A\mathbf{h}\|_2}{\|\mathbf{h}\|_2} = \max_{\|\mathbf{h}\|_2 \leq 1} \|A\mathbf{h}\|_2,$$

which is equivalent to the largest singular value of A

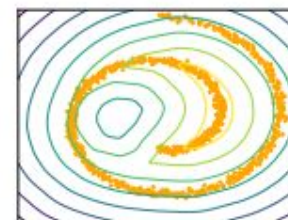
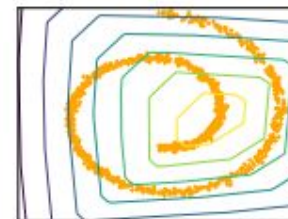
8 Gaussians



25 Gaussians



Swiss Roll

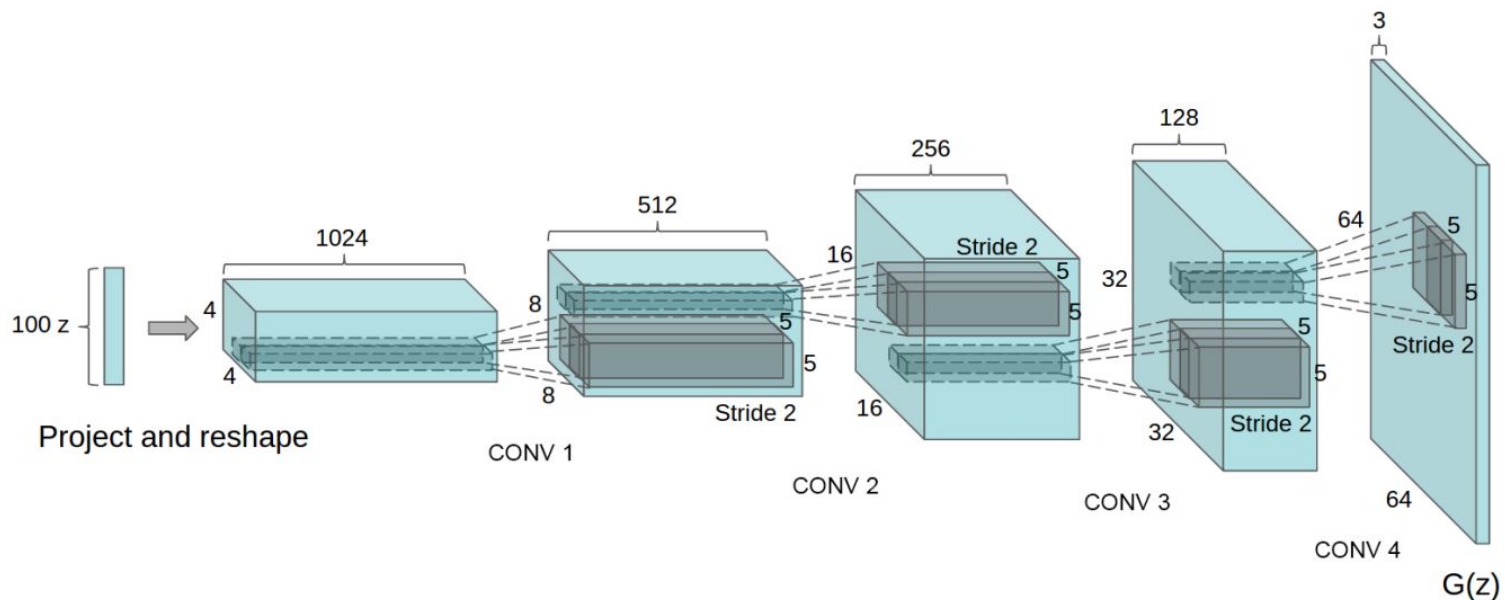


Value surfaces of WGAN critics trained to optimality on toy datasets using (top) weight clipping and (bottom) gradient penalty. Critics trained with weight clipping fail to capture higher moments of the data distribution. The ‘generator’ is held fixed at the real data plus Gaussian noise.

GAN Applications

DCGAN

- Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
- Use batchnorm in both the generator and the discriminator.
- Remove fully connected hidden layers for deeper architectures.
- Use ReLU activation in generator for all layers except for the output, which uses Tanh.
- Use LeakyReLU activation in the discriminator for all layers.

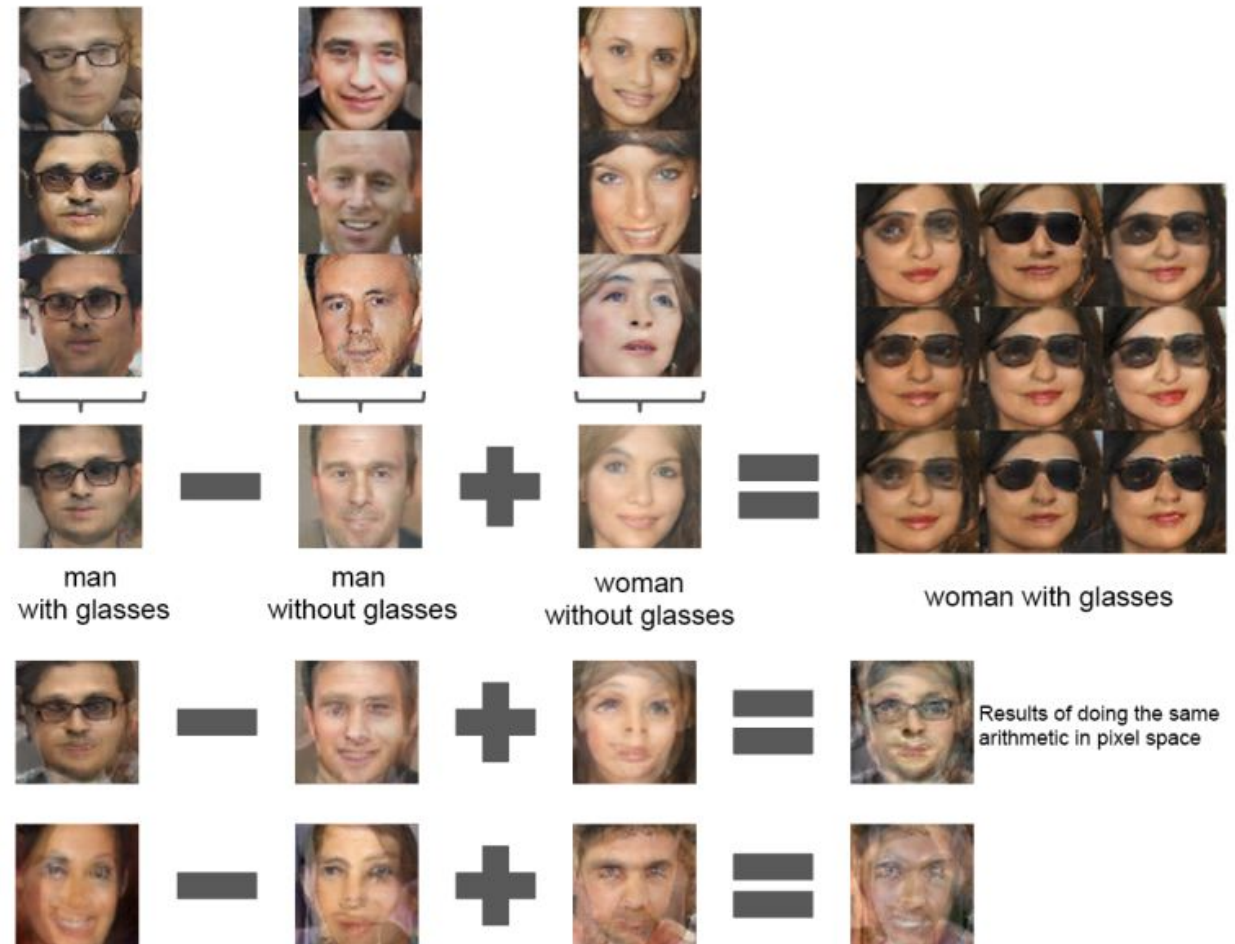


DCGAN generator used for LSUN scene modeling. A 100 dimensional uniform distribution Z is projected to a small spatial extent convolutional representation with many feature maps. A series of four fractionally-strided convolutions (in some recent papers, these are wrongly called deconvolutions) then convert this high level representation into a 64×64 pixel image. Notably, no fully connected or pooling layers are used.

DCGAN. Vector arithmetic

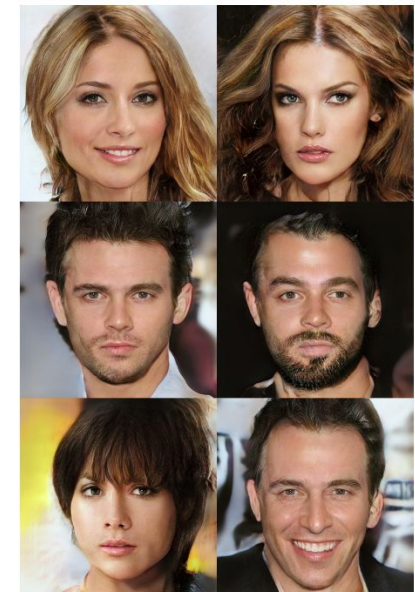
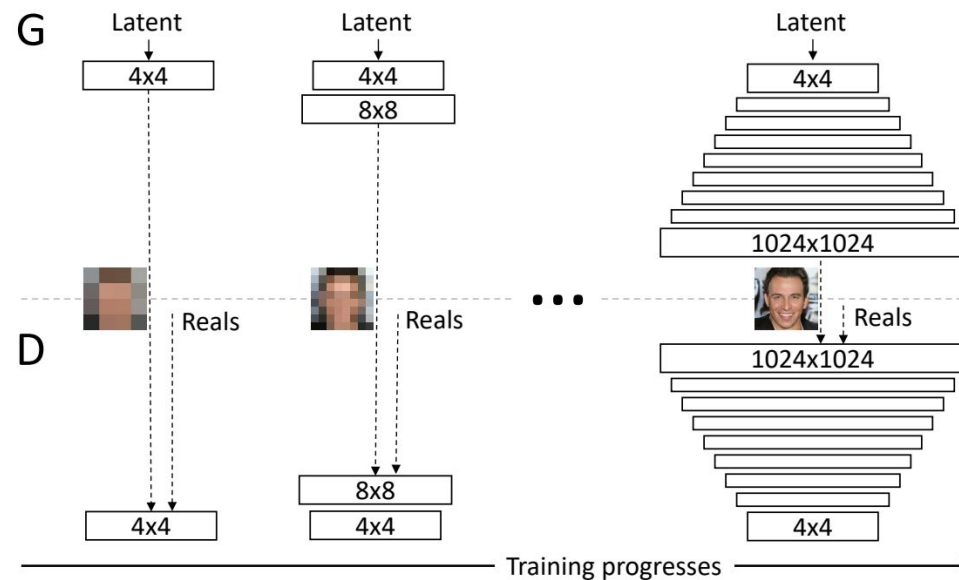
- For each column, the Z vectors of samples are averaged.
- Arithmetic was then performed on the mean vectors creating a new vector Y.
- The center sample on the right hand side is produced by feeding Y as input to the generator.
- To demonstrate the interpolation capabilities of the generator, uniform noise sampled with scale ± 0.25 was added to Y to produce the 8 other samples.
- Applying arithmetic in the input space (bottom two examples) results in noisy overlap due to misalignment.

[Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks](#)



Progressive Growing GAN

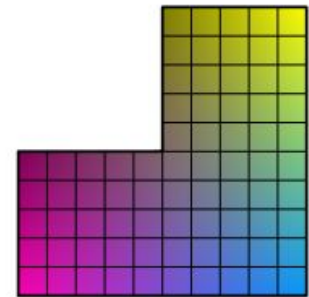
- Grow both the generator and discriminator progressively
- Starting from a low resolution, add new layers that model increasingly fine details as training progresses:
 - Train GAN which generate $N \times N$ images
 - Increase N
 - Add upsampling layers to G, downsampling layers to D
 - Repeat
- Speeds the training up and greatly stabilizes it



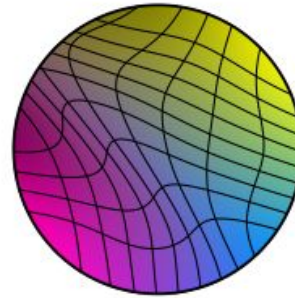
[Progressive Growing of GANs for Improved Quality, Stability, and Variation](#)

Mapping Network

- We expect each component of input vector $\mathbf{Z}$ to be responsible for some particular generative factor.
- Introduce Mapping network $\mathcal{W} = f(\mathbf{Z})$ to reduce correlations between components of $\mathbf{Z}$.
- (a) An example training set where some combination (e.g., long haired males) is missing.
- (b) This forces the mapping from $\mathbf{Z}$ to image features to become curved so that the forbidden combination disappears in $\mathbf{Z}$ to prevent the sampling of invalid combinations.
- (c) The learned mapping from $\mathbf{Z}$ to $\mathcal{W}$ is able to “undo” much of the warping.



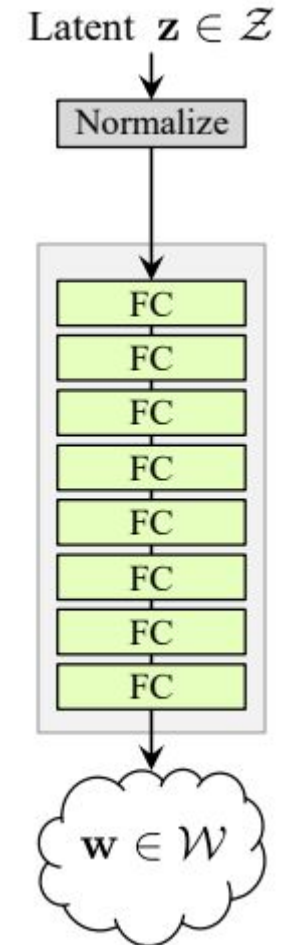
(a) Distribution of features in training set



(b) Mapping from $\mathcal{Z}$ to features



(c) Mapping from $\mathcal{W}$ to features



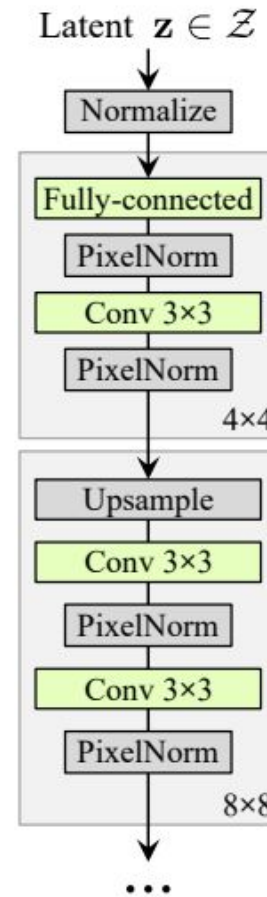
StyleGAN

- Map the input to an intermediate latent space W
- Learned affine transformations specialize W to styles $y = (y_s, y_b)$
- It controls the generator through adaptive instance normalization (AdaIN) at each convolution layer:

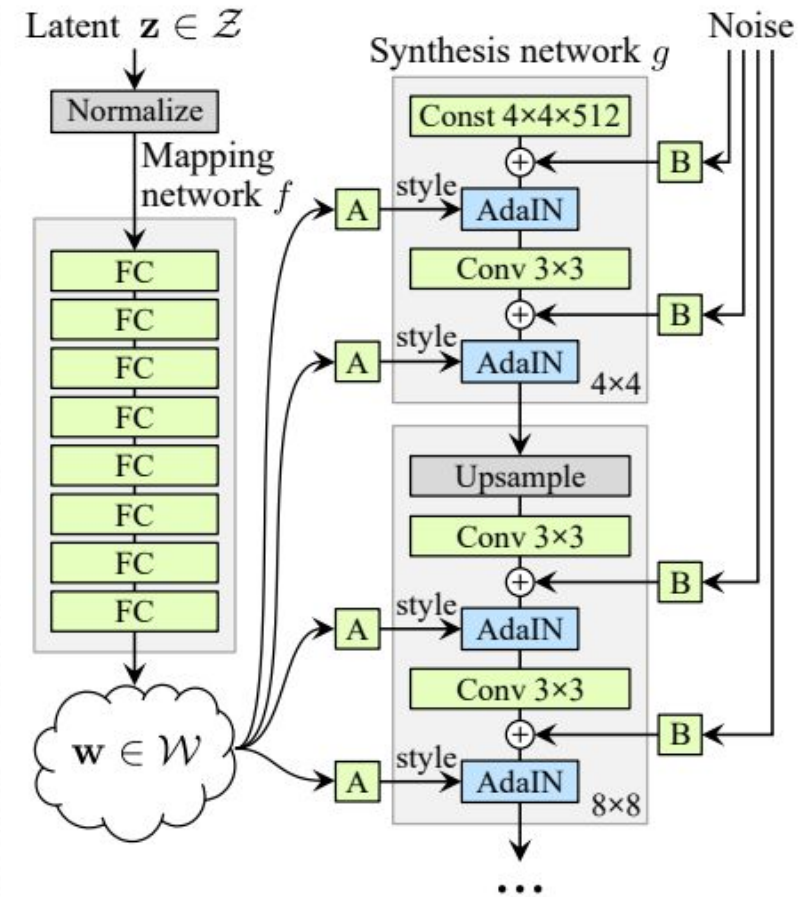
$$\text{AdaIN}(\mathbf{x}_i, \mathbf{y}) = \mathbf{y}_{s,i} \frac{\mathbf{x}_i - \mu(\mathbf{x}_i)}{\sigma(\mathbf{x}_i)} + \mathbf{y}_{b,i},$$

- Gaussian noise is added after each convolution, before evaluating the nonlinearity.
- “A” stands for a learned affine transform, and “B” applies learned per-channel scaling factors to the noise input.
- The output of the last layer is converted to RGB using a separate 1×1 convolution.

[A Style-Based Generator Architecture for Generative Adversarial Networks](#)

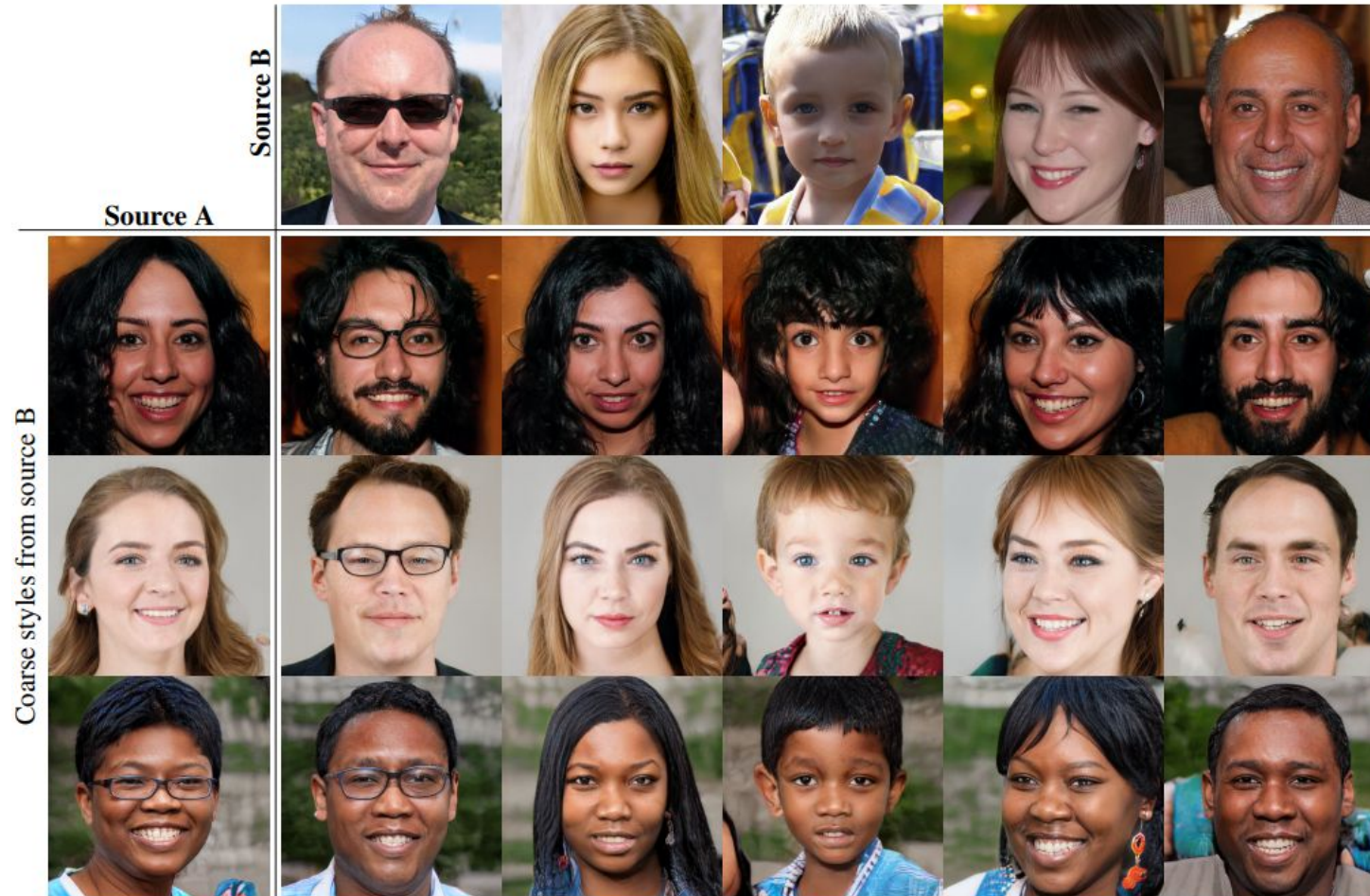


(a) Traditional



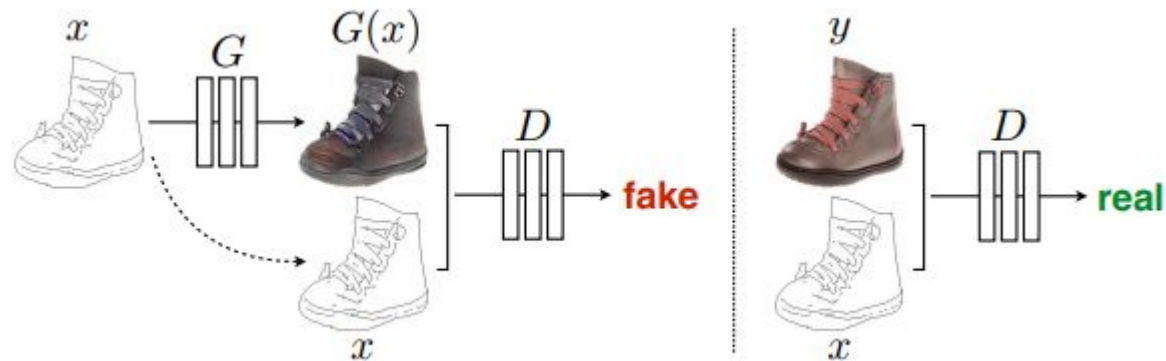
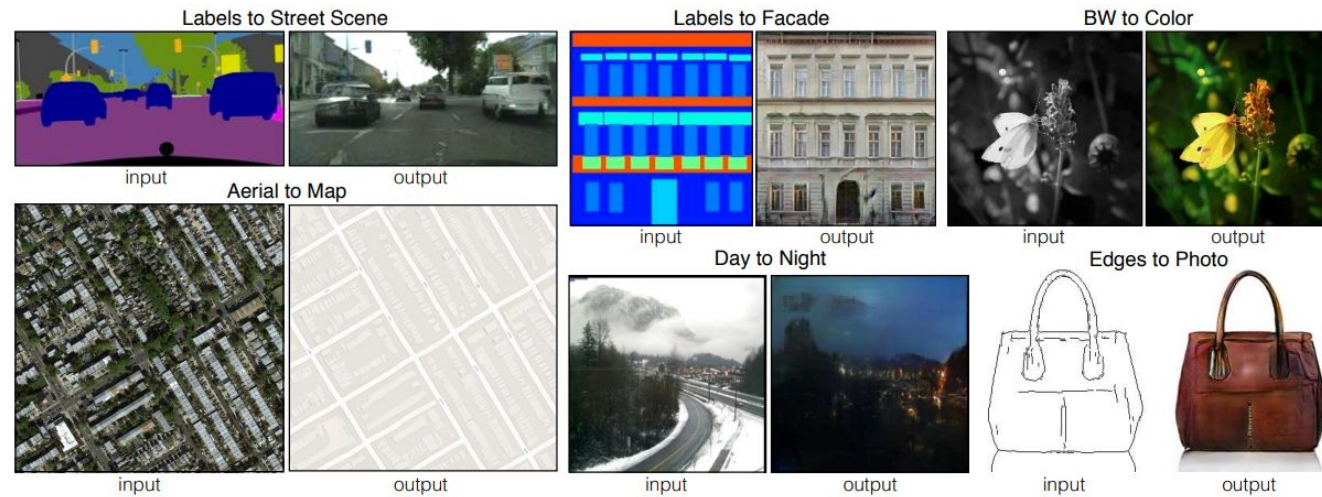
(b) Style-based generator

StyleGAN. Results



[A Style-Based Generator Architecture for Generative Adversarial Networks](#)

Image-to-Image with CGAN



[Image-to-Image Translation with Conditional Adversarial Networks](#)

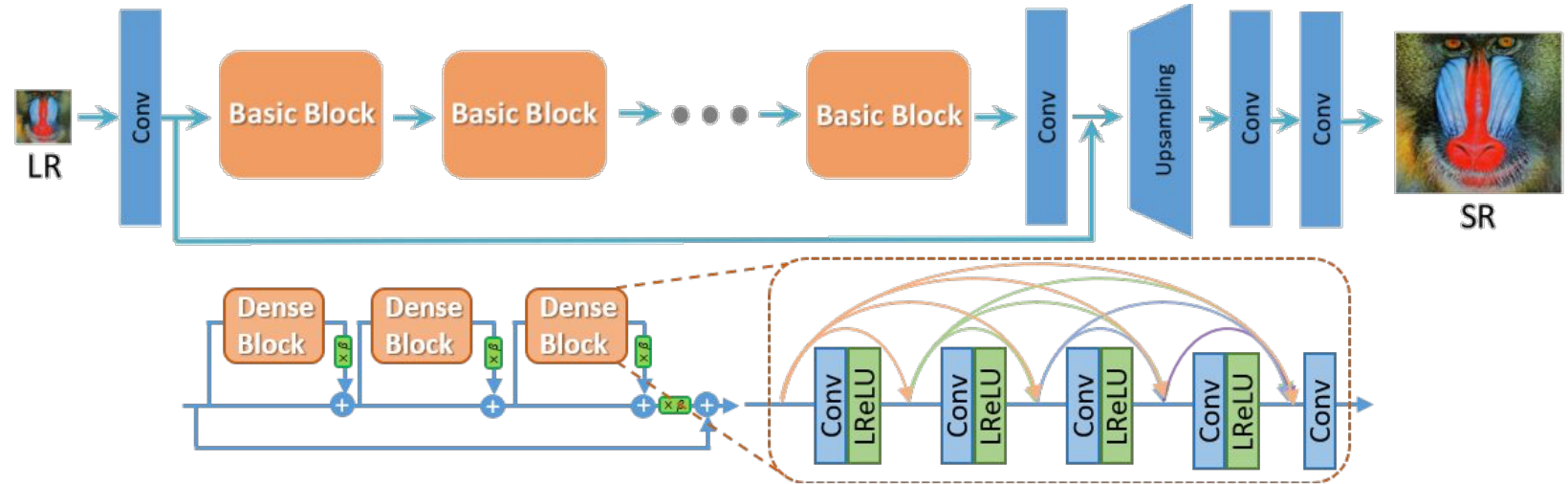
Image-to-Image with CGAN. Results



[Image-to-Image Translation with Conditional Adversarial Networks](#)

Super Resolution. ESRGAN

- SRResNet-based architecture with residual-in-residual blocks
- Removing BN layers has proven to increase performance and reduce computational complexity in different PSNR-oriented tasks including SR
- Mixture of context, perceptual, and adversarial losses
- Context and perceptual losses are used for proper image upscaling
- Adversarial loss pushes neural network to the natural image manifold using a discriminator network that is trained to differentiate between the super-resolved images and original photo-realistic images
- Different from the standard discriminator D in SRGAN, which estimates the probability that one input image x is real and natural, a relativistic discriminator tries to predict the probability that a real image is relatively more realistic than a fake one



$$D(x_r) = \sigma(C(\text{Real})) \rightarrow 1 \text{ Real?}$$

$$D(x_f) = \sigma(C(\text{Fake})) \rightarrow 0 \text{ Fake?}$$

a) Standard GAN

➔

$$D_{Ra}(x_r, x_f) = \sigma(C(\text{Real}) - \mathbb{E}[C(\text{Fake})]) \rightarrow 1 \text{ More realistic than fake data?}$$

$$D_{Ra}(x_f, x_r) = \sigma(C(\text{Fake}) - \mathbb{E}[C(\text{Real})]) \rightarrow 0 \text{ Less realistic than real data?}$$

b) Relativistic GAN

[ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks](#)

ESRGAN Results



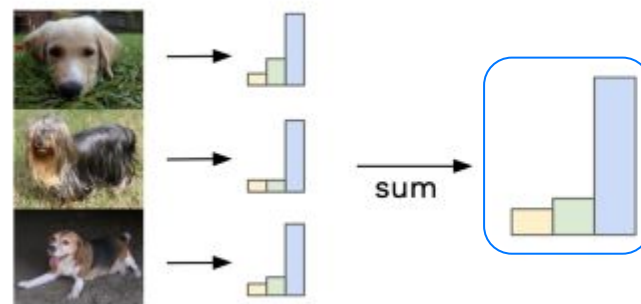
[ESRGAN: Enhanced Super-Resolution
Generative Adversarial Networks](#)

GAN Evaluation

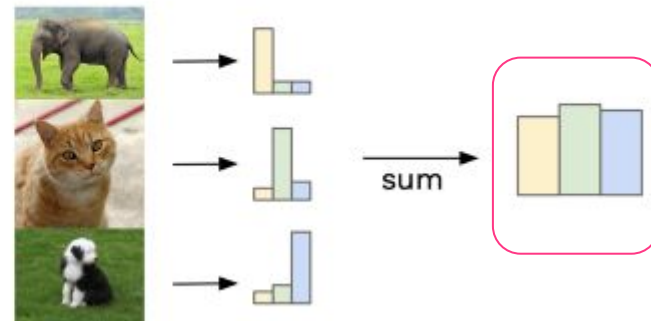
Inception Score

- Correlates well with human evaluation of image quality
- The score measures two things simultaneously:
 - The images have variety (diversity)
 - Each image distinctly looks like something (sharpness)
- Summing the label distributions of images, we create a new label distribution, the marginal distribution
- The marginal distribution tells us how much variety there is in our generator's output
- $IS = \exp(\mathbb{E}_{\mathbf{x}} \text{KL}(\underbrace{p(y|\mathbf{x})}_{\text{Label distribution}} \parallel \underbrace{p(y)}_{\text{Marginal distribution}}))$

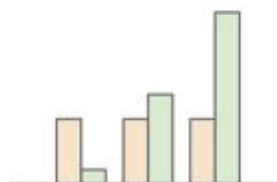
Similar labels sum to give focussed distribution



Different labels sum to give uniform distribution

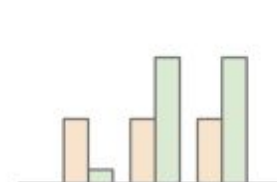


High KL divergence



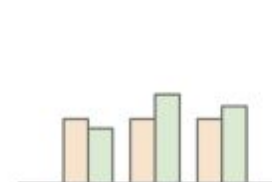
Ideal situation

Medium KL divergence



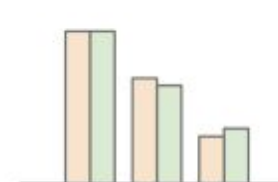
Generated images are not distinctly one label

Low KL divergence

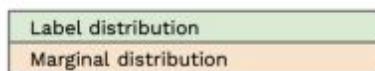


Generated images are not distinctly one label

Low KL divergence

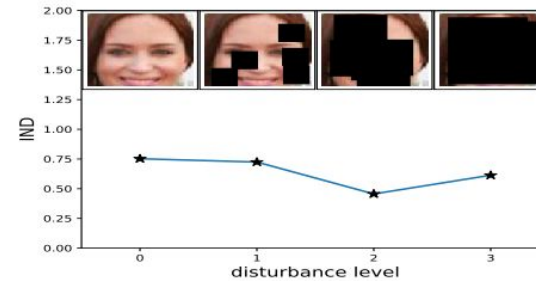
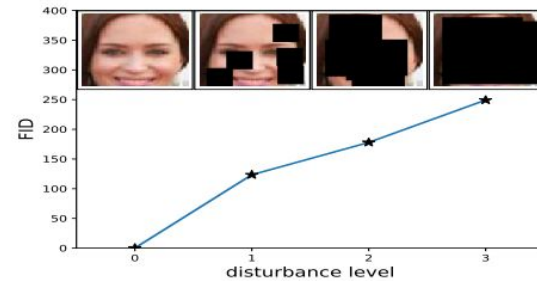
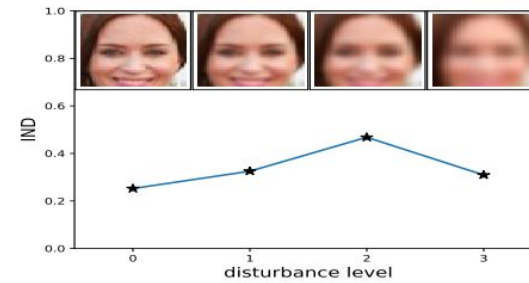
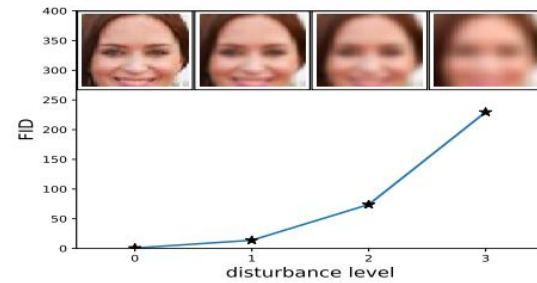
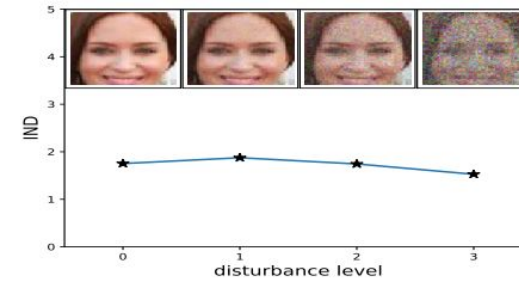
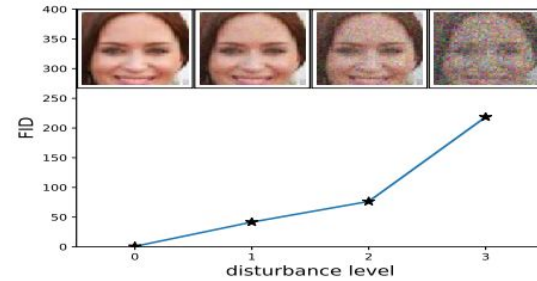


Generator lacks diversity



Frechet Inception Distance

$$FID = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$



[GANs Trained by a Two Time-Scale Update Rule](#)
[Converge to a Local Nash Equilibrium](#)

Precision-Recall

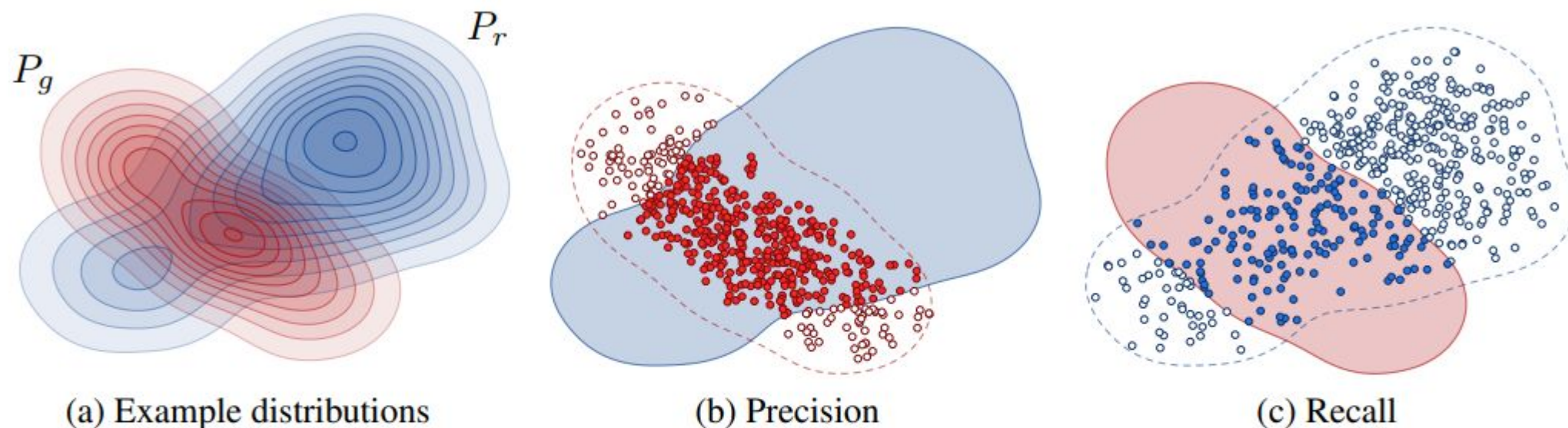


Figure 1: Definition of precision and recall for distributions [25]. (a) Denote the distribution of real images with P_r (blue) and the distribution of generated images with P_g (red). (b) Precision is the probability that a random image from P_g falls within the support of P_r . (c) Recall is the probability that a random image from P_r falls within the support of P_g .

Precision-Recall

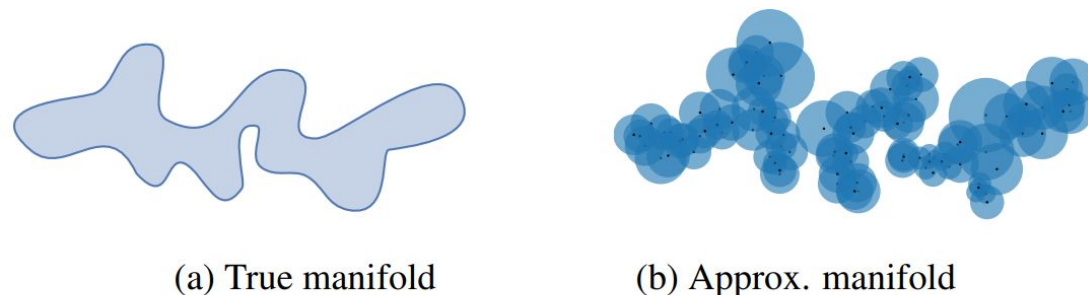


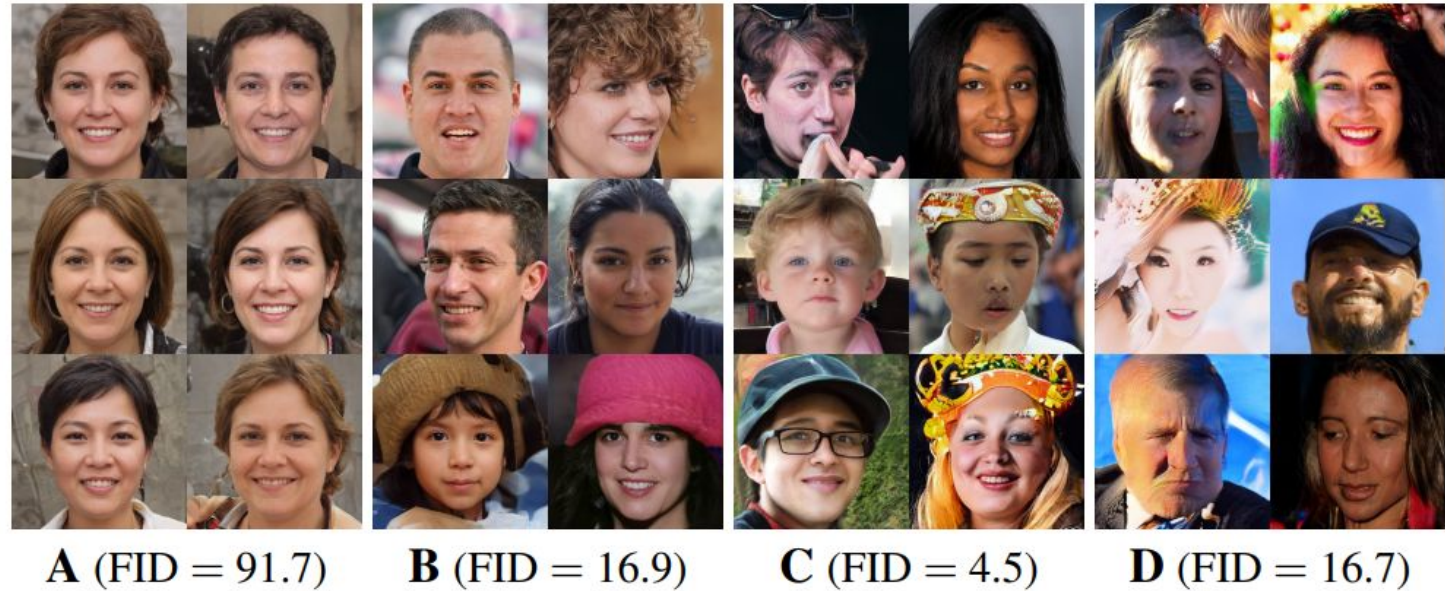
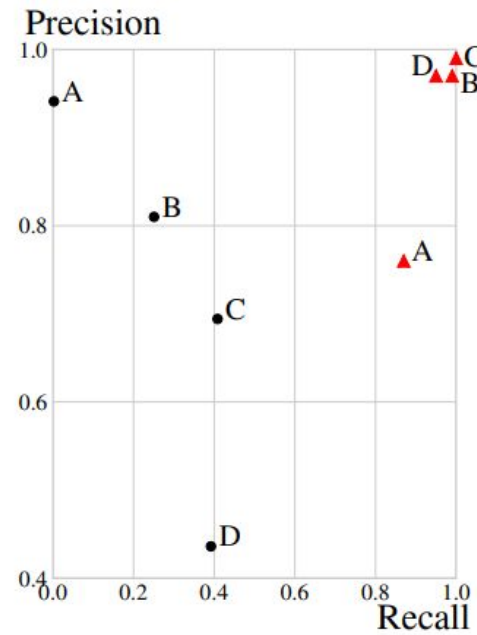
Figure 2: (a) An example manifold in a feature space. (b) Estimate of the manifold obtained by sampling a set of points and surrounding each with a hypersphere that reaches its k th nearest neighbor.

$$f(\phi, \Phi) = \begin{cases} 1, & \text{if } \|\phi - \phi'\|_2 \leq \|\phi' - \text{NN}_k(\phi', \Phi)\|_2 \text{ for at least one } \phi' \in \Phi \\ 0, & \text{otherwise,} \end{cases} \quad (1)$$

where $\text{NN}_k(\phi', \Phi)$ returns k th nearest feature vector of ϕ' from set Φ . In essence, $f(\phi, \Phi_r)$ provides a way to determine whether a given image looks realistic, whereas $f(\phi, \Phi_g)$ provides a way to determine whether it could be reproduced by the generator. We can now define our metric as

$$\text{precision}(\Phi_r, \Phi_g) = \frac{1}{|\Phi_g|} \sum_{\phi_g \in \Phi_g} f(\phi_g, \Phi_r) \quad \text{recall}(\Phi_r, \Phi_g) = \frac{1}{|\Phi_r|} \sum_{\phi_r \in \Phi_r} f(\phi_r, \Phi_g) \quad (2)$$

Precision-Recall VS FID



[Improved Precision and Recall Metric for Assessing Generative Models](#)

Further reading

- [Pros and Cons of GAN Evaluation Measures](#)
- [Pros and Cons of GAN Evaluation Measures: New Developments](#)
- [Large Scale GAN Training for High Fidelity Natural Image Synthesis](#)
- [Why Spectral Normalization Stabilizes GANs: Analysis and Improvements](#)
- [Alias-Free Generative Adversarial Networks \(StyleGAN3\)](#)
- [Towards Real-World Blind Face Restoration with Generative Facial Prior](#)
- [StyleCLIP: Text-Driven Manipulation of StyleGAN Imagery](#)

